EXERCICE 10 · CHANGEMENTS ENTRE LES ÉTAPES

Todo App

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Point de départ** · template (4 fichiers, en entier)
2. **Étape 1 → 2** · correction (2 fichiers)

Départ Point de départ : template

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg, package.json

DÉPART **src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

DÉPART **src/App.jsx** 8 lignes

```jsx
import "./App.css";
import Todo from "./components/Todo";

function App() {
  return <Todo />;
}

export default App;
```

DÉPART **src/App.css** 32 lignes

```css
/* VERSION RESPONSIVE - S'adapte à tous les écrans (mobile, tablette, desktop) */

/* Importation de la police Google Fonts Inter avec plusieurs graisses */
@import url("https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap");

/* Sélecteur universel qui cible tous les éléments HTML */
* {
  /* Force tous les éléments à inclure padding et border dans leur largeur/hauteur totale */
  box-sizing: border-box;
  /* Supprime les marges par défaut du navigateur pour avoir un contrôle total */
  margin: 0;
  /* Supprime les espacements internes par défaut du navigateur */
  padding: 0;
}

/* Style global du corps de la page */
body {
  /* Applique la police Inter importée, avec sans-serif comme police de secours */
  font-family: "Inter", sans-serif;
  /* Améliore le rendu des polices sur les navigateurs WebKit (Safari, Chrome) */
```

```
  -webkit-font-smoothing: antialiased;
  /* Améliore le rendu des polices sur Firefox pour Mac */
  -moz-osx-font-smoothing: grayscale;

  text-align: center;

  padding: 20px;
}

h1 {
  padding: 20px;
}
```

DÉPART `src/components/Todo.jsx` — 44 lignes

```jsx
import { useEffect, useState } from "react";

// Exercice 10 · template : la Todo App faite ensemble (10 todos, sans loading ni filtre)
// TODO 1 : les filtres All / Todo / Done, avec le nombre de todos sur chaque bouton
// TODO 2 : un state loading, et afficher « Loading... » pendant le chargement
// TODO 3 : une checkbox devant chaque todo pour la cocher / décocher (completed)
// TODO 4 : une todo terminée est barrée, avec une opacité plus faible

export default function TodoApp() {
  const [todos, setTodos] = useState([]);

  // Fonction de fetch
  const fetchTodos = async () => {
    try {
      const res = await fetch(
        "https://jsonplaceholder.typicode.com/todos?_limit=10"
      );
      const data = await res.json();
      setTodos(data);
    } catch (error) {
      console.error("Error:", error);
    }
  };

  // Un seul useEffect pour charger les données
  useEffect(() => {
    fetchTodos();
  }, []); // Une seule fois au mount

  return (
    <div style={{ padding: 16, maxWidth: 600, margin: "0 auto" }}>
      <h2>Todo App</h2>

      {/* inline-block : la liste est centrée dans la page, le texte reste à gauche */}
      <ul style={{ padding: 0, display: "inline-block", textAlign: "left" }}>
        {todos.map((todo) => (
          <li key={todo.id} style={{ listStyle: "none", margin: 12 }}>
            {todo.title}
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Étape 1 → 2 template → correction

Les filtres All / Todo / Done avec leur nombre, le Loading, la checkbox pour cocher une todo, et la todo terminée barrée.

MODIFIÉ `package.json` +1 −1

```
@@ -1,5 +1,5 @@
 {
-  "name": "exercice-10-template",
+  "name": "vite-project",
   "private": true,
   "version": "0.0.0",
   "type": "module",
```

MODIFIÉ `src/components/Todo.jsx` +105 −14

```
@@ -1,13 +1,23 @@
 import { useEffect, useState } from "react";
-
-// Exercice 10 · template : la Todo App faite ensemble (10 todos, sans loading ni filtre)
-// TODO 1 : les filtres All / Todo / Done, avec le nombre de todos sur chaque bouton
-// TODO 2 : un state loading, et afficher « Loading... » pendant le chargement
-// TODO 3 : une checkbox devant chaque todo pour la cocher / décocher (completed)
-// TODO 4 : une todo terminée est barrée, avec une opacité plus faible

 export default function TodoApp() {
   const [todos, setTodos] = useState([]);
+  const [filter, setFilter] = useState("all");
+  const [loading, setLoading] = useState(true);
+
+  // Calcul dérivé au lieu d'un état séparé
+  const getFilteredTodos = (todos, filter) => {
+    switch (filter) {
+      case "todo":
+        return todos.filter((t) => !t.completed);
+      case "done":
+        return todos.filter((t) => t.completed);
+      default:
+        return todos;
+    }
+  };
+
+  const filteredTodos = getFilteredTodos(todos, filter);

   // Fonction de fetch
   const fetchTodos = async () => {
@@ -19,6 +29,8 @@
       setTodos(data);
     } catch (error) {
       console.error("Error:", error);
+    } finally {
+      setLoading(false);
     }
   };

@@ -27,18 +39,97 @@
     fetchTodos();
   }, []); // Une seule fois au mount

+  // useEffect avec dépendance
+  useEffect(() => {
+    console.log("Filter changed:", filter);
```

```
+  }, [filter]);
+
+  // Toggle pour modifier les todos
+  const toggleTodo = (id) => {
+    /*const nextTodos = [...todos];
+    const todo = nextTodos.find((t) => t.id === id);
+    if (todo) {
+      todo.completed = !todo.completed;
+      setTodos(nextTodos);
+    }*/
+
+    /*setTodos(
+      todos.map((t) => (t.id === id ? { ...t, completed: !t.completed } : t))
+    );*/
+
+    setTodos((prevTodos) =>
+      prevTodos.map((t) =>
+        t.id === id ? { ...t, completed: !t.completed } : t
+      )
+    );
+  };
+
   return (
     <div style={{ padding: 16, maxWidth: 600, margin: "0 auto" }}>
       <h2>Todo App</h2>

-      {/* inline-block : la liste est centrée dans la page, le texte reste à gauche */}
-      <ul style={{ padding: 0, display: "inline-block", textAlign: "left" }}>
-        {todos.map((todo) => (
-          <li key={todo.id} style={{ listStyle: "none", margin: 12 }}>
-            {todo.title}
-          </li>
-        ))}
-      </ul>
+      {/* Filtres avec compteurs */}
+      <div
+        style={{
+          display: "flex",
+          gap: 8,
+          justifyContent: "center",
+          margin: 16,
+        }}
+      >
+        <button
+          onClick={() => setFilter("all")}
+          style={{ fontWeight: filter === "all" ? "bold" : "normal" }}
+        >
+          All ({todos.length})
+        </button>
+        <button
+          onClick={() => setFilter("todo")}
+          style={{ fontWeight: filter === "todo" ? "bold" : "normal" }}
+        >
+          Todo ({todos.filter((t) => !t.completed).length})
+        </button>
+        <button
+          onClick={() => setFilter("done")}
+          style={{ fontWeight: filter === "done" ? "bold" : "normal" }}
+        >
+          Done ({todos.filter((t) => t.completed).length})
+        </button>
+      </div>
+
+      {/* Liste ou Loading */}
+      {loading ? (
```

```
+         <div style={{ textAlign: "center" }}>Loading...</div>
+       ) : (
+         <ul style={{ padding: 0 }}>
+           {filteredTodos.map((todo) => (
+             <li key={todo.id} style={{ listStyle: "none", margin: 12 }}>
+               <label
+                 style={{
+                   cursor: "pointer",
+                   display: "flex",
+                   alignItems: "center",
+                 }}
+               >
+                 <input
+                   type="checkbox"
+                   checked={todo.completed}
+                   onChange={() => toggleTodo(todo.id)}
+                   style={{ marginRight: 8 }}
+                 />
+                 <span
+                   style={{
+                     textDecoration: todo.completed ? "line-through" : "none",
+                     opacity: todo.completed ? 0.6 : 1,
+                   }}
+                 >
+                   {todo.title}
+                 </span>
+               </label>
+             </li>
+           ))}
+         </ul>
+       )}
       </div>
     );
   }
```